

PROGRAMAÇÃO DE
COMPUTADORES I

A PRIMEIRA CONVERSA COM A MÁQUINA

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- Sair da teoria e começar a dar ordens reais ao computador.

A ILUSÃO DA INTELIGÊNCIA

O computador não é **inteligente**; ele é apenas incrivelmente **rápido** e **obediente**. Para que ele faça algo **útil**, precisamos estabelecer uma **via de comunicação dupla**.

OS TRÊS PASSOS BÁSICOS DE UM PROGRAMA

- Ele precisa te escutar (Entrada de Dados).
- Ele precisa pensar (Processamento).
- Ele precisa te responder (Saída de Dados).

SAÍDA DE DADOS

- O Comando:
print()
- É a **voz** do computador. Ele pega uma **informação** que está escondida na **memória** e exibe na **tela** para o usuário ler.

SAÍDA DE DADOS

Como usar: Tudo o que você quer que ele diga, exatamente como está escrito, deve estar entre aspas.

SAÍDA DE DADOS

Exemplo de Código:

- `print("Olá! Sistema iniciado com sucesso.")`

VARIÁVEIS

- **Variáveis** são as "**Caixas**" na Memória que podem armazenar **informação**.
- Antes de pedir **informações** ao usuário, precisamos de um lugar para **guardar** essas **informações**.

VARIÁVEIS

- **O que é uma variável?** Imagine um gigantesco **armazém** (a memória **RAM** do computador) cheio de caixas **vazias**.
- Uma **variável** é quando você pega uma dessas **caixas**, cola uma **etiqueta** com um **nome** nela e coloca uma **informação** dentro.

VARIÁVEIS

Exemplo no Código:

```
1.nome_do_cliente = "Maria"
```

```
2.idadeDoCliente = 20
```

Regra de Ouro: O **nome** da caixa, da sua variável, **não** pode ter **espaços**! Usamos o **underline** (**_**) ou letras **maiúsculas** para separar as palavras.

ENTRADA DE DADOS

○ Comando:

input()

- O que faz? Ele pausa o programa, exibe uma pergunta na tela e fica esperando o usuário digitar uma resposta e apertar a tecla "Enter".

ENTRADA DE DADOS

A Regra da Caixa: Você nunca usa o **input()** sozinho. A **resposta** do usuário “cairia no chão” e se perderia. Você sempre precisa **guardar** a resposta em uma **variável** (caixa).

ENTRADA DE DADOS

Exemplo de Código:

```
1.nome_do_cliente = input("Por favor, digite  
o seu nome: ")  
2.print("Bem-vindo(a),", nome_do_cliente)
```


A NECESSIDADE DOS TIPOS DE DADOS

O Computador é Literal: Se você tentar **somar** a palavra "**Maçã**" com o número **5**, um humano entende que é uma **brincadeira** ou um **erro**. O computador entra em **pânico** e o programa **trava**.

A NECESSIDADE DOS TIPOS DE DADOS

Ele precisa saber exatamente qual é o **tipo** de **informação** que está dentro da “**caixa**” (variável) para saber quais operações **matemáticas** ou **lógicas** ele tem **permissão** para fazer.

Surgem os Tipos de Dados Fundamentais!!!

OS TIPOS DE DADOS BÁSICOS DO PYTHON

Texto (String / str): Letras, palavras, **frases**, ou até números que você não vai usar para fazer contas (como CEP ou CPF). Sempre vem entre aspas duplas.

- Exemplo: "Engenharia", "123.456.789-00", "Sim"

OS TIPOS DE DADOS BÁSICOS DO PYTHON

Número Inteiro (int): Números redondos, positivos ou negativos, **sem casas decimais**. Usado para contar coisas inteiras.

- Exemplo: 45, -10, 0, 2026

OS TIPOS DE DADOS BÁSICOS DO PYTHON

Booleano (bool): O interruptor. Só aceita dois valores: Verdadeiro ou Falso. Usado para lógica.

- True, False

O PROBLEMA DO INPUT()

A Regra Cega do Python: Tudo o que o usuário digita no **input()** entra no computador como **Texto** (str), mesmo que ele digite um número!

O PROBLEMA DO INPUT()

Se o usuário digitar **10**, o computador enxerga a **palavra** "10", não o número. Se você tentar multiplicar por 2, ele não vai dar 20, ele vai repetir o texto: "1010".

O PROBLEMA DO INPUT()

A Solução (**Conversão**): Precisamos **avisar** ao **computador** para **transformar** o texto em número no momento em que ele entra.

1. idade = int(input("Digite sua idade: "))
(Garante que será um número inteiro).
2. peso = float(input("Digite seu peso: "))
(Garante que será um número com casas decimais).

ALGORITMO ESTRUTURADO

- Vamos criar um programa útil: Uma calculadora de desconto para uma loja.

ALGORITMO ESTRUTURADO

Passo 1: Entradas (Ouvir)

1. `preco_original = float(input("Qual o valor do produto? "))`
- 2.
3. `desconto = float(input("Qual a porcentagem de desconto? "))`

ALGORITMO ESTRUTURADO

Passo 2: Processamento (Pensar)

1. $\text{valor_do_desconto} = \text{preco_original} * (\text{desconto} / 100)$
- 2.
3. $\text{preco_final} = \text{preco_original} - \text{valor_do_desconto}$

ALGORITMO ESTRUTURADO

Passo 3: Saída (Falar)

1. `print("O valor do desconto em reais é:",
valor_do_desconto)`
- 2.
3. `print("O valor final a ser pago é:",
preco_final)`

PROVOCAÇÃO FINAL

O algoritmo que **acabamos** de construir em minutos para calcular um **desconto** tem exatas **6** linhas. Se uma loja tem **10.000** produtos e precisa aplicar descontos na Black Friday, um ser humano com uma **calculadora** levaria **dias**. O computador, rodando esse mesmo código de 6 linhas **repetidas vezes***, faz isso em **menos de 1 segundo****.

PROVOCAÇÃO FINAL

Na sua **futura** área de **atuação**, você imagina que existam **tarefas matemáticas** ou **burocráticas repetitivas** que roubam **tempo** dos **profissionais** e que poderiam ser **resolvidas** com poucas linhas de código?

DÚVIDAS???